reward, we will naively assume their equivalent actions, but if we compare to what we expected would happen, and one reward performed much better than anticipated, that action should be reinforced more.

As with the rest of the deep learning methods, we generally must use batches of data in order to effectively train. Training with a single example a time introduces too much noise, and the training will likely never converge. To introduce batch training with Q-learning we used an experience replay buffer that could randomly select batches of previous experiences. We could have used experience replay with actor-critic, but it is more common to use distributed training with actor-critic (and, to be clear, Q-learning can also be distributed). Distributed training in actor-critic models is more common because we often want to use a recurrent neural network (RNN) layer as part of our reinforcement learning model in cases where keeping track of prior states is necessary or helpful in achieving the goal. But RNNs need a sequence of temporally related examples, and experience replay relies on a batch of independent experiences. We could store entire trajectories (sequences of experiences) in a replay buffer, but that just adds complexity. Instead, with distributed training and each process running online with its own environment, the models can easily incorporate RNNs.

We didn't cover it here, but there's another way to train an online actor-critic algorithm besides distributed training: simply utilize multiple copies of your environment, and then batch together the states from each independent environment, feeding it into a single actor-critic model that will then produce independent predictions for each environment. This is a viable alternative to distributed training when the environments are not expensive to run. If your environment is a complicated, high-memory- and computer-intensive simulator, it's probably going to be very slow to run multiple copies of it in a single process, so in that case a distributed approach is better.

We have now covered what we consider to be the most foundational parts of reinforcement learning today. You should now be comfortable with the basic mathematical framework of reinforcement learning as a Markov decision process (MDP), and you should be able to able to implement Q-